Multi-tab work and getting data out

Week 7, Lecture 2 · Browser Use: How LLM Agents Drive the Web

Summer 2026

What we'll cover

- Opening and switching tabs: the built-in tab actions
- Shared login state across tabs in one session
- Downloads: accept_downloads, downloads_path, and what happens to the file
- Recording runs with video and HAR for debugging
- The decision rule: built-in actions vs custom tool

By the end of this lecture you can build an agent that spans two tabs, saves a downloaded file, and records the run for later review.

Multi-tab work

Tabs share the browser context

Within one `BrowserSession`, every tab shares:

- The same cookie jar
- The same localStorage
- The same login state

A login that happened on tab 1 is immediately present on tab 2.
This is the core property that makes multi-tab agents useful for authenticated work.

Tab actions in the built-in registry

The agent has access to tab-management actions in the tools registry.
As of mid-2026, the built-in action set includes actions in the family of:

- **Open a new tab** at a given URL
- **Switch to a tab** by index or by page content description

The agent can emit both in a single step if the task permits multi-act.

```
task = """
1. Open https://example.com/inbox, note the unread count.
2. Open a second tab to https://example.com/compose.
3. Return: the unread count from step 1.
"""
```

The agent carries the count in the `memory` field of AgentOutput across steps.

Write task prompts by content, not by index

Tab indices are not stable. If the agent opens tabs in a different order than intended, index 0 may not be the tab you expect.

Fragile	Better
"Switch to tab 0"	"Switch to the inbox tab"
"Go back to tab 1"	"Return to the page showing unread messages"
"Use tab 2 for the download"	"On the exports page, click download"

Describe the tab by what is on it, not by its position in the tab bar.

Downloads and file handling

Three parameters control downloads

```
from browser_use.browser import BrowserSession

session = BrowserSession(
    accept_downloads=True,
    downloads_path="/tmp/agent-dl",
    auto_download_pdfs=True,   # PDF links save instead of opening
)
```

Source: browser-use team, Browser Settings docs (2026)

When the agent clicks a download link:

1. The browser saves the file to `downloads_path`
2. The agent receives confirmation that the download completed
3. The file is on disk, the agent does not move or process it

What happens after the download

The built-in actions stop at the click. For anything after that, write a custom tool.

```python
from browser_use import Tools
from browser_use.agent.views import ActionResult
from pydantic import BaseModel
import csv, pathlib

tools = Tools()
DL = pathlib.Path("/tmp/agent-dl")

class ParseCSVInput(BaseModel):
    filename: str

@tools.action("Parse a downloaded CSV and return the row count")
async def parse_csv(params: ParseCSVInput) -> ActionResult:
    path = DL / params.filename
    with open(path) as f:
        rows = list(csv.reader(f))
    return ActionResult(extracted_content=f"{len(rows) - 1} data rows")
```

Recording runs for debugging

Video and HAR recording

```
session = BrowserSession(
    record_video_dir="/tmp/agent-videos",
    record_video_size={"width": 1280, "height": 900},
    record_video_framerate=10,
    record_har_path="/tmp/run.har",
)
```

Source: browser-use team, Browser Settings docs (2026)

- **Video**: play it back to see exactly what the agent saw at each step
- **HAR (HTTP Archive)**: open in Chrome DevTools Network tab → Import HAR
- **Playwright traces** (`traces_dir`): the most detailed format; use `playwright show-trace`

When to record and when not to

Situation	Record?
New workflow, first run	Yes, always
Diagnosing a failing agent	Yes
Production, agent runs reliably	No, recording uses CPU
Sharing a demo with a colleague	Yes (video only)

Recording is heavier than a normal run. Disable it once the agent works.

Built-in actions vs custom tools

The decision rule

Ask: can this step be done by clicking, typing, scrolling, or extracting text from a page?

- **Yes** → built-in actions handle it
- **No** → write a custom tool

Common custom tool cases from this week:

Task	Why custom tool?
Write downloaded CSV to a database	Browser cannot INSERT into Postgres
Parse a PDF	Browser opens PDF viewer; no text extraction built in
Call a backend API directly	Not a browser action at all
Rename or move a downloaded file	Filesystem, not browser

The handoff pattern

```
task = """
1. Go to https://reports.example.com/export (you are already logged in).
2. Click 'Download monthly report'. The file is called summary.csv.
3. Call summarize_report with filename='summary.csv'.
4. Return the result.
"""
```

Steps 1-2: built-in navigate + click actions.
Step 3: the registered custom tool.
Step 4: the agent returns the `ActionResult` content as the final answer.

The task prompt drives the handoff. The agent calls the tool when it matches the task description.

Take-aways

1. All tabs in one `BrowserSession` share the same cookie jar; a login on tab 1 is present on tab 2 without any additional auth step.
2. Write tab prompts by page content, not tab index. Tab indices shift when tabs are opened in unexpected orders.
3. `accept_downloads=True` and `downloads_path` route files to disk. Built-in actions stop at the download click; a custom tool handles the file.
4. Record with `record_video_dir` and `record_har_path` while debugging; turn recording off once the agent runs reliably.
5. The decision rule for custom tools: if the step cannot be done through a browser interaction, write a tool.

> "Custom action handlers can request injected dependencies by parameter name: browser_session, page_extraction_llm, file_system, available_file_paths."

browser-use contributors, *browser-use/browser-use* (GitHub, 2026)

What's next

- **Section (this week):** Persist a login once, run a second agent across two tabs, hands-on with the full pattern
- **Capstone (assigned this week):** Most capstone tasks need auth or downloads. Decide today which your project needs.
- **Week 8:** Making agents reliable, loop detection, max_failures, and task framing for the hard cases
- **HW4:** Harden an authenticated agent (out this week)